

WP-6 Angular Foundation & Auth

Weeks: Week 5 Track: Frontend

Goal. A clean Angular app that authenticates against your real backend and protects its routes — the shell the booking UI lives in.

Tasks

- ☐ Set up the Angular app with standalone components and sensible routing.
- ☐ Build login; store and refresh tokens correctly on the client.
- ☐ Add an HTTP interceptor that attaches auth and handles token refresh.
- ☐ Add route guards so unauthenticated users can't reach protected pages.
- ☐ Establish a state-management approach and stick to it.
- ☐ Handle API errors gracefully in the UI.

Acceptance criteria

- ✓ A user logs in through the UI and reaches an authenticated area.
- ✓ Protected routes are inaccessible without a valid session.
- ✓ Token refresh happens transparently via the interceptor.
- ✓ API errors surface as clear user feedback, not silent failures.

WP-7 Booking UI & Calendar

Weeks: Weeks 5–6 Track: Frontend

Goal. The screens a real user touches: browse resources, see availability, book, and manage bookings — including a calendar that survives real data.

Tasks

- ☐ Resource list and detail views.
- ☐ Availability view for a resource and date range.
- ☐ Booking form for one-off and recurring bookings, with clear validation feedback.
- ☐ Calendar view rendering bookings, including recurring series, without choking on volume.
- ☐ Approval queue UI for approvers.
- ☐ Cancellation and blackout handling in the UI.
- ☐ Wire the full flow end-to-end against the real API.

HARD PROBLEM The calendar must stay responsive with hundreds of bookings and expanded recurring series. Naive rendering will melt — think about what you fetch and what you render.

Acceptance criteria

- ✓ A member completes browse → book → confirm entirely through the UI.
- ✓ Recurring bookings render correctly in the calendar.
- ✓ The calendar stays responsive under realistic data volume.
- ✓ An approver can action pending requests from the UI.